
CyLag user manual

Release 1.0

Fabien Souillé

Oct 07, 2026

CONTENTS:

1	Quick installation	1
1.1	Requirements	1
1.2	Compile and import CyLag	1
1.3	Repository	2
1.4	Build documentation	2
1.5	Setup a simulation and run CyLag	3
2	Mathematical models	5
2.1	Overview	5
2.2	Lagrangian Stochastic Model n°1 (LSM-1)	8
2.3	Lagrangian Stochastic Model n°2 (LSM-2)	10
2.4	Lagrangian Stochastic Model n°3 (LSM-3)	14
2.5	Pseudo-3D velocity corrections	16
2.6	Particle Modules	17
3	Particle-mesh implementation	19
3.1	General overview of the algorithm	19
3.2	Interpolation	19
3.3	Search algorithm	22
3.4	Boundary conditions	22
4	Setup, inputs and practical informations	25
4.1	Setup philosophy	25
4.2	Overview of input parameters	27
4.3	Adding particles to a simulation	31
4.4	Saving particles	34
4.5	Post-processing particle files	34
5	Good practices	35
5.1	Mesh and Eulerian fields quality	35
5.2	Number of particles	35
5.3	Choice of numerical parameters	35
5.4	I/O frequency	36
6	Developer guide	37
6.1	Get access	37
6.2	Development checklist	37
6.3	Unit tests	38
6.4	Performance considerations and profiling	38

QUICK INSTALLATION

1.1 Requirements

The following packages are required to install and use CyLag: *cython* ≥ 0.29 , *numpy*, *scipy*, *mpi4py*, *h5py*, *pytest*, *matplotlib*, *setuptools*, *sphinx*, and *sphinx-gallery* (for the documentation).

If you already use [openTELEMAC](#), all Python requirements should already be installed (except Sphinx). If not, the recommended practice is to install a python virtual environment:

```
python3 -m venv pyenv/cylag
```

After creation, activate venv :

```
source pyenv/cylag/bin/activate
```

Install all requirements :

```
pip install -r REQUIREMENTS.txt
```

1.2 Compile and import CyLag

CyLag is primarily implemented in [Cython](#) and [Python](#), combining the performance of compiled code with the flexibility of a high-level scripting language. Before using CyLag, the library must be compiled. Compilation is performed by executing the `setup.py` script located in the root directory of the repository. Once compiled, CyLag can be imported and used as a standard Python package.

- Add CyLag to the Python path by copying the following line into `~/.bashrc`:

```
export PYTHONPATH=$PYTHONPATH:</path/to/cylag>
```

- Compile CyLag:

```
python3 setup.py build_ext --inplace
```

- Compile CyLag in debug mode:

```
python3 setup.py build_ext --inplace --debug
```

- Test importing CyLag:

```
python3
import cylag
```

- When recompiling the code, you might want to clean the previously built extensions by using:

```
python3 setup.py clean
```

1.3 Repository

The CyLag repository is organized into several directories, including:

- `cylag`: source code of the library;
- `examples`: validation test cases and benchmark simulations;
- `tests`: unit tests used to verify implementation and non-regression;
- `docs`: Sphinx documentation.

1.4 Build documentation

The CyLag repository also includes a dedicated Sphinx-based documentation, providing installation instructions, theoretical background, API references, tutorials, and practical examples to facilitate the use and development of the code.

- Install Sphinx:

```
pip install sphinx sphinx-gallery sphinx-autodoc-typehints
```

- Build the documentation (html):

```
cd docs
make html
```

The documentation should be available in `docs/build/html/index.html`.

- Build the documentation (latex/pdf):

```
cd docs
make latexpdf
```

Note

the gallery of example and API are not included with the latex build of the documentation.

1.5 Setup a simulation and run CyLag

CyLag follows a **component-based architecture** with clear separation of concerns. To set up a simulation, the user needs to define three main objects:

- **EulerianFieldSet**: contains the Eulerian fields computed with a 2D or 3D hydrodynamic solver such as TELEMAC,
- **LagrangianParticleSet**: contains the particles modeled by CyLag and their physical properties,
- **Solver**: solves the Lagrangian Stochastic Models and updates the particle state vector. It takes **EulerianFieldSet** and **LagrangianParticleSet** as inputs, as well as a **parameters** dictionary containing the main solver options.

CyLag is run using its Python API; to launch a simulation, one has to define a Python script in which `cylag` is imported and `cylag` objects are defined. The simulation script usually looks like this:

```
import cylag
fset = cylag.EulerianFieldSet(...)
pset = cylag.LagrangianParticleSet(...)
parameters = {...}
cylag_solver = cylag.Solver(fset, pset, parameters)
cylag_solver.solve()
```

Each object has its own inputs and parameters. See the gallery of [examples/](#) for a more in-depth overview of CyLag's possibilities. Also see [Setup, inputs and practical informations](#) for a complete breakdown of the input parameters (and make sure you read the theoretical aspects to fully understand them).

MATHEMATICAL MODELS

2.1 Overview

CyLag implements several general Lagrangian formulations, in 2D and 3D, allowing the framework to accommodate a wide range of applications through the adjustment of closure relations associated with the forces acting on particles (e.g., drag, buoyancy, and additional forces), distribution of particle sizes, choice of numerical integration schemes, as well as through different diffusion parameterizations.

Note

In the following, the parameters of the models that can be modified in CyLag are tagged as follows:

- `fset:parameter`: passed to the `EulerianFieldSet` object,
- `pset:parameter`: passed to the `LagrangianParticleSet` object,
- `param:parameter`: passed to the `parameters` dictionary.

Lagrangian Stochastic Models

Three Lagrangian Stochastic Models (LSMs) of increasing complexity are available, distinguished by the definition of the particle state vector. The simplest formulation, LSM-1, considers only the particle position (denoted $\mathbf{x}_p$) and describes transport through an advection-diffusion process. LSM-2 extends the state vector by introducing the particle velocity (denoted $\mathbf{U}_p$), thereby accounting for inertial effects through Newton's equation of motion. Finally, LSM-3 includes the fluid velocity seen by the particle (denoted $\mathbf{U}_s$) in addition to its position and velocity and relies on a generalized Langevin formulation to represent turbulent velocity fluctuations and their temporal correlations. Together, these models provide a flexible framework for simulating a broad spectrum of transported substances, ranging from passive tracers to inertial particles in turbulent free-surface flows. In this section, the governing equations associated with these three model formulations are presented.

- *LSM-1 (Lagrangian Stochastic Model n°1)*: one equation (position only).
- *LSM-2 (Lagrangian Stochastic Model n°2)*: two equations (position and velocity).
- *LSM-3 (Lagrangian Stochastic Model n°3)*: three equations (position, velocity, and fluid velocity seen).

Numerical methods

Numerical integration methods differ depending on whether the governing equations constitute a system of ordinary differential equations (ODEs) in the absence of diffusion or a system of stochastic differential equations (SDEs). The numerical integration of SDEs involves stochastic integrals, which preclude the direct application of classical deterministic methods developed for ODEs, such as high-order Runge-Kutta schemes. Consequently, both the mathematical formulation of the problem and the nature of the governing equations strongly influence the choice of the numerical integration method. Table 1 summarizes the different model formulations implemented in CyLag, together with their underlying assumptions and corresponding numerical schemes.

Table 1: Description of CyLag Lagrangian models and associated numerical schemes. Euler-Maruyama (EM), Euler (E), second-order (RK2), third order (RK3) fourth-order (RK4) Runge-Kutta, and Direct Integration (DI).

Model	State vector	Diffusion	Model name	Equations	Schemes
LSM-1	$\{\mathbf{x}_p\}$	Yes	Random Displacement Model	SDE	EM
LSM-1	$\{\mathbf{x}_p\}$	No	Material path-lines	ODE	E, RK2, RK3, RK4
LSM-2	$\{\mathbf{x}_p, \mathbf{U}_p\}$	Yes	Langevin model	SDE	EM, DI
LSM-2	$\{\mathbf{x}_p, \mathbf{U}_p\}$	No	Newton model	ODE	E, RK2, RK3, RK4, DI
LSM-3	$\{\mathbf{x}_p, \mathbf{U}_p, \mathbf{U}_s\}$	Yes	Generalized Langevin	SDE	EM, DI

Dimension

The Lagrangian Stochastic Models in CyLag can be solved using 2D ($\mathbf{x}_p, \mathbf{U}_p, \mathbf{U}_s \in \mathbb{R}^2$) or 3D particles ($\mathbf{x}_p, \mathbf{U}_p, \mathbf{U}_s \in \mathbb{R}^3$). While 2D particles can only be used with 2D Eulerian field set the 3D particles can be used with either 2D or 3D field set. The selection of the model is done by modifying the value of the `pset:dim` and `fset:dim` parameters. The available options are as follows :

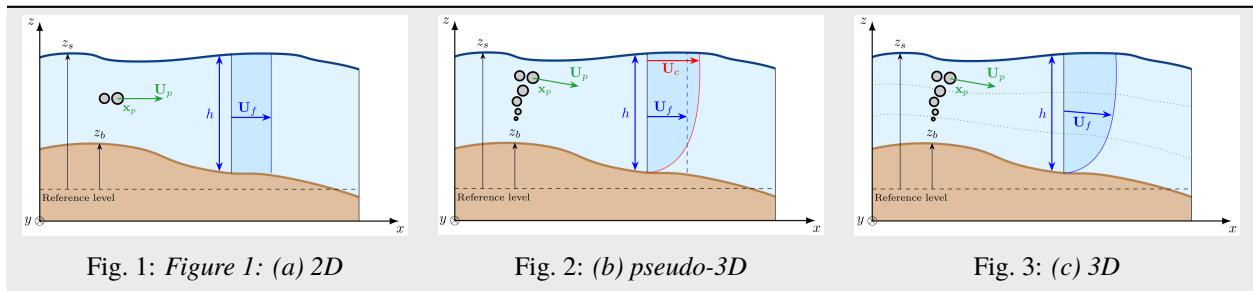
- *2D simulation*: `pset:dim=2` with `fset:dim=2`;
- *3D simulation*: `pset:dim=3` with `fset:dim=3`;
- *Pseudo-3D simulation*: `pset:dim=3` with `fset:dim=2`.

Note

The models used for hydrodynamics are not detailed here, for more information please refer to the [TELEMAC](#) theory guides.

In Figure 1, we illustrate of the different dimensions of the `LagrangianParticleSet` and `EulerianFieldSet` objects, as well as the main notations. In the following, we denote z_b the bottom elevation, z_s the free surface elevation, $h = z_s - z_b$ the water depth and $\mathbf{U}_f$ the mean fluid velocity, which can be either: $\mathbf{U}_f = (u_f, v_f) \in \mathbb{R}^2$ or: $\mathbf{U}_f = (u_f, v_f, w_f) \in \mathbb{R}^3$ depending on `fset.dim`.

Table 2: Illustration of `LagrangianParticleSet` and `EulerianFieldSet` dimensions and main notations :



Particle distribution

CyLag supports both mono-disperse and poly-disperse particle set. In the first case all particles are assumed to have the same size while the second option accounts for the natural variability in particle size. For poly-disperse modeling, CyLag supports several particle-diameter distributions (see Figure 2). User needs to provide the mean particle diameter `pset.particle_diameter` and a distribution `pset.particle_diameter_distribution` which is a list containing the distribution type and main parameter (e.g. `['uniform', 0.5]`). The options are as follows:

- **None** : mono-disperse particle set, all particles have the same size equal to `pset.particle_diameter`;
- **[uniform, Δ]** : $d_p \sim U(\mu - \Delta, \mu + \Delta)$ with `pset.particle_diameter` = μ , Δ = half-width;
- **[normal, σ]** : $d_p \sim N(\mu, \sigma^2)$ with `pset.particle_diameter` = μ , σ = standard deviation;
- **[lognormal, GSD]** : $\ln(d_p) \sim N(\mu_l, \sigma_l^2)$ with `pset.particle_diameter` = $E[d_p]$, $e^{\sigma_l} = GSD$;
- **[weibull, k]** : $d_p \sim \text{Wei}(k, \lambda)$ with $\lambda = E[d_p]/\Gamma(1 + 1/k)$, `pset.particle_diameter` = $E[d_p]$, k = shape.
- **[tabulated, None]** : user-defined tabulated distribution, with the first column containing the particle diameters and the second column containing the corresponding cumulative distribution function (CDF) values. The tabulated distribution must be defined in `cylag/core/particle_distribution.pyx` and CyLag must be re-compiled.

These distributions enable the representation of both symmetric and skewed particle-size populations, providing flexibility for modelling a wide range of particle types.

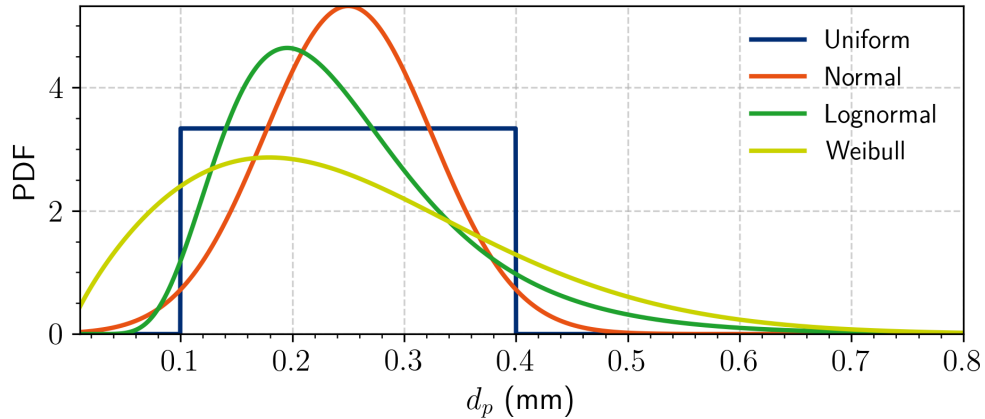


Fig. 4: Figure 2: Example of particle distributions available in CyLag.

Note

To prevent nonphysical diameters and avoid artificial concentrations at the bounds, we use truncation by rejection sampling: values outside the prescribed diameter range ($<1.e-12$, >10) are discarded and redrawn. Note that this truncation modifies the distribution's mean, variance, and higher moments when many samples are rejected.

2.2 Lagrangian Stochastic Model n°1 (LSM-1)

The LSM-1 formulation is the simplest model implemented in CyLag. The particle state vector is restricted to the particle position, and particle motion is described directly through an advection-diffusion equation. In this framework, particles are transported by the mean flow field, while unresolved turbulent fluctuations are represented by a stochastic displacement term. As a result, LSM-1 belongs to the family of Random Displacement Models (RDMs) and is particularly well suited for tracer-like particles whose inertia can be neglected. Owing to its simplicity and low computational cost, it is commonly used for large-scale transport and dispersion studies.

1. Particle state vector: $\{\mathbf{x}_p\}$ (position only)

2. Main equation: The evolution of a particle's position is computed as:

$$dx_{p,i} = (U_{f,i} + U_{b,i} + U_{a,i} + \partial_i K_i) dt + \sqrt{2K_i} dW_i,$$

where:

- $i \in \{1, 2, 3\}$ indicates the axis x, y, z respectively,
- $\mathbf{x}_p$ is the particle's position,
- $\mathbf{U}_f$ is the mean fluid velocity interpolated at the particle's position,
- $\mathbf{U}_b = (0, 0, -w_p)$ is the buoyancy velocity (only for 3D),
- $\mathbf{U}_a$ is an additional velocity (user-defined or module-dependent),
- $\mathbf{W}$ is a Wiener process, whose increment $d\mathbf{W} = \mathbf{W}(t + dt) - \mathbf{W}(t)$ is a Gaussian variable with zero mean and variance dt (see Figure 4),
- K_i ($0 \leq i \leq 2$) are horizontal diffusivity coefficients and K_3 is the vertical diffusivity coefficient.

3. Buoyancy models (set via `pset:buoyancy_velocity_model`)

- **Model 0:** constant buoyancy velocity defined via the parameter `pset:buoyancy_velocity` (default = 0.)
- **Model 1:** computed within CyLag (see Figure 3) as follows:

$$w_p = \frac{gd_p^2}{18\nu_f}(s-1), \text{ if } d_p \leq 10^{-4},$$

$$w_p = \frac{10\nu_f}{d_p} \left(\sqrt{1 + \frac{(s-1)gd_p^3}{100\nu_f^2}} - 1 \right), \text{ if } 10^{-4} < d_p \leq 10^{-3},$$

$$w_p = \sqrt{\frac{4d_pg(s-1)}{3C_d}}, \text{ if } d_p \geq 10^{-3},$$

where $s = \rho_p/\rho_f$, and

- g is the gravitational acceleration (set at $g = 9.80665$),
- ρ_f is the water density (`param:water_density`, default: 1000),
- ρ_p is the particle density (`pset:particle_density`, default: 1000),
- d_p is the particle diameter (`pset:particle_diameter`, default: 0.01)

4. Diffusion models (set via `param:diffusion_model`)

- **Model 0:** no diffusion, $K_i = 0$ ($0 \leq i \leq 3$).

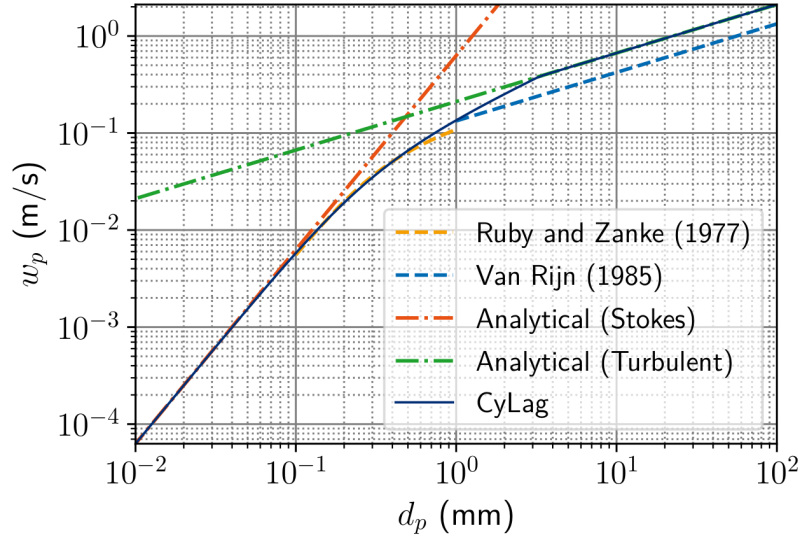


Fig. 5: Figure 3: Buoyancy velocity analytical solution (CyLag LSM-1).

- **Model 1:** constant isotropic diffusivity defined as $K_1 = K_2$ (param:horizontal_diffusivity) and K_3 (param:vertical_diffusivity).
- **Model 2:** $k - \varepsilon$ turbulence model for the horizontal component and constant vertical diffusivity; the horizontal diffusivity is then defined as $K_i = \nu_t / \sigma_c$ ($0 \leq i \leq 2$), in which σ_c is the Schmidt number (param:schmidt_number, default: 0.72) and ν_t is the turbulent diffusivity defined as $\nu_t = C_\mu k^2 / \varepsilon$, where C_μ is the Prandtl-Kolmogorov constant equal to $C_\mu = 0.09$.
- **Model 3:** $k - \varepsilon$ turbulence model for both horizontal and vertical components, with $K_i = \nu_t / \sigma_c$ ($0 \leq i \leq 3$).

Note

When using the $k - \varepsilon$ turbulence model, the mean Eulerian fields k and ε should be passed to the EulerianFieldSet object with the optional_fields parameter.

Note

The stochastic terms require an RNG (Random Number Generator) to generate the Wiener process increment $d\mathbf{W}$. The RNG can be changed via the parameter param:rng_method which can have two values:

- param:rng_method=1: rectangular approximation with mean=0, var=1;
- param:rng_method=2: Ziggurat method (default);
- param:rng_method=3: Marsaglia's polar method.

All methods use the PCG32 as a base for pseudorandom number generator.

Table 3: Illustration of stochastic diffusion in 1D and 2D

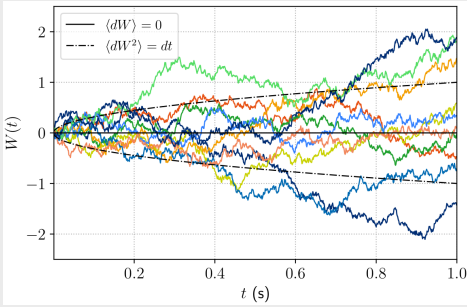


Fig. 6: Figure 4: (a) 1D (10 particles)

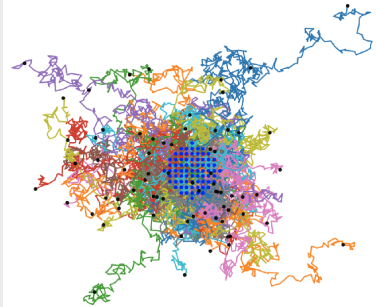


Fig. 7: (b) 2D (70 particles)

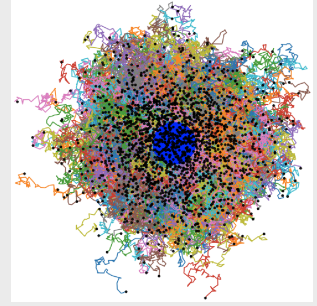


Fig. 8: (b) 2D (2000 particles)

5. Time integration schemes (set via `param:time_scheme`)

- `param:time_scheme=1`: **Euler** (1st order) or **Euler-Maruyama** (1st order stochastic);
- `param:time_scheme=2`: **Runge-Kutta 2** (2nd order);
- `param:time_scheme=3`: **Runge-Kutta 3** (3rd order);
- `param:time_scheme=4`: **Runge-Kutta 4** (4th order).

Warning

High order anticipating methods such as the **Runge-Kutta** schemes are only suitable for ODE systems, ensure the diffusion is negligible before using these schemes.

2.3 Lagrangian Stochastic Model n°2 (LSM-2)

The LSM-2 formulation extends the state vector by introducing the particle velocity in addition to the particle position. Particle motion is therefore governed by both a displacement equation and a momentum equation derived from Newton's second law. This approach allows the model to account for particle inertia and forces such as drag, buoyancy, added mass, pressure-gradient effects, and user-defined external forces. Compared with LSM-1, LSM-2 is better suited for modeling sediments, floating debris, biological particles, or any substance whose velocity may differ significantly from the local fluid velocity.

1. Particle state vector: $\{\mathbf{x}_p, \mathbf{U}_p\}$ (position, velocity)

2. Main equations: Newton's second law of motion is solved for every particle, leading to a set of two equations for the update of the position and velocity. Two variants are available in CyLag, depending on whether diffusion is applied to velocity or position (see `param:diffusion_lsm2_option`):

- **Diffusion on position** (`param:diffusion_lsm2_option=1`):

$$\begin{aligned} dx_{p,i} &= (U_{p,i} + \partial_i K_i) dt + \sqrt{2K_i} dW_i, \\ dU_{p,i} &= A_0 g_i dt + A_1 (U_{f,i} - U_{p,i}) dt + A_2 dU_{f,i} + A_3 F_{a,i} dt, \end{aligned}$$

- **Diffusion on velocity** (`param:diffusion_lsm2_option=2` or `3`):

$$\begin{aligned} dx_{p,i} &= U_{p,i} dt, \\ dU_{p,i} &= A_0 g_i dt + A_1 (U_{f,i} - U_{p,i}) dt + A_2 dU_{f,i} + A_3 F_{a,i} dt + B_{p,i} dW_i, \end{aligned}$$

where:

- $i \in \{1, 2, 3\}$ indicates the axis x, y, z respectively,
- $\mathbf{g} = (0, 0, -g)$ is the gravitational acceleration (set at $g = 9.80665$),
- $\mathbf{x}_p$ is the particle's position,
- $\mathbf{U}_p$ is the particle velocity,
- $\mathbf{U}_f$ is the mean fluid velocity interpolated at particle's position,
- $\mathbf{F}_a$ is an additional force (user-defined or module-dependent),
- $\mathbf{W}$ is a Wiener process, whose increment $\mathbf{W} = \mathbf{W}(t + dt) - \mathbf{W}(t)$ is a Gaussian variable with zero mean and variance dt ,
- K_i ($0 \leq i \leq 2$) are horizontal diffusivity coefficients and K_3 is the vertical diffusivity coefficient,
- $B_{p,i}$ ($0 \leq i \leq 2$) are horizontal velocity diffusivity coefficients and $B_{p,3}$ the vertical velocity diffusivity coefficient.

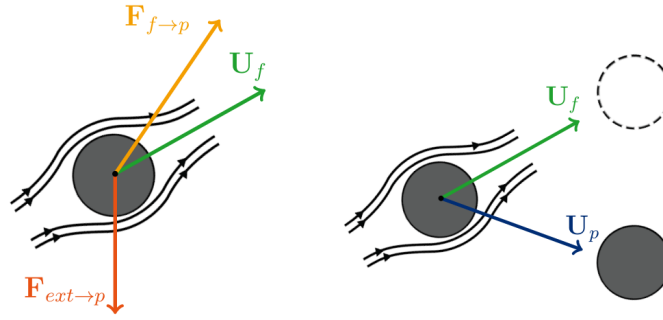


Fig. 9: *Figure 5: Force balance on a spherical particle in a fluid (left) considering external forces such as gravity and fluid forces; and trajectory separation illustration (right). U_f stands for the mean fluid velocity, with which fluid material particles are transported (dotted circle), whereas U_p stands for the particle velocity resulting from Newton's second law of motion and with which solid particles are transported (filled circle).*

The coefficients $\mathcal{A}_i$ are defined by considering the balance of the following forces acting on a particle: *buoyancy, drag, added mass, and pressure gradient*:

- $\mathcal{A}_0 = \frac{(\rho_p - \rho_f)}{\rho_p + \rho_f C_a}$,
- $\mathcal{A}_1 = \frac{\rho_f S_p C_d |\mathbf{U}_f - \mathbf{U}_p|}{2(\rho_p + \rho_f C_a) V_p}$,
- $\mathcal{A}_2 = \frac{\rho_f (1 + C_a)}{\rho_p + \rho_f C_a}$,
- $\mathcal{A}_3 = \frac{1}{(\rho_p + \rho_f C_a) V_p}$,

where

- ρ_f is the water density (param:water_density, default : 1000),
- ρ_p is the particle density (pset:particle_density, default : 1000),
- d_p is the particle diameter (pset:particle_diameter, default : 0.01)
- S_p is the apparent surface of the particle, defined as $S_p = \pi d_p^2/4$,
- V_p is the particle volume, given by $V_p = \pi d_p^3/6$,

- C_a is the added mass coefficient (`pset:added_mass_coef`, default: 0.5),
- C_d is the drag coefficient, defined hereafter.

The *added mass* and *pressure gradient* forces can be deactivated (`pset:added_mass_force=False`), in which case we have

- $\mathcal{A}_0 = \frac{(\rho_p - \rho_f)}{\rho_p},$
- $\mathcal{A}_1 = \frac{\rho_f S_p C_d |\mathbf{U}_f - \mathbf{U}_p|}{2\rho_p V_p},$
- $\mathcal{A}_2 = 0,$
- $\mathcal{A}_3 = \frac{1}{\rho_p V_p}.$

Note

There is no need to model **buoyancy** with analytical formulas in LSM-2, as it is natively included in the model by taking gravity and drag forces into account.

3. Drag models: different drag models are available for the drag coefficient C_d via the parameter `pset:drag_coefficient_model` (see Figure 6):

- **Model 0:** constant coefficient defined via the `pset:drag_coefficient` parameter,
- **Model 1:** Schiller (1933):

$$C_d = \begin{cases} \frac{24}{Re_p} (1 + 0.15 Re_p^{0.687}) & \text{if } Re_p \leq 1000 \\ 0.44 & \text{if } Re_p > 1000 \end{cases}$$

where $Re_p = d_p |\mathbf{U}_f - \mathbf{U}_p| / \nu_f$, where ν_f is the fluid kinematic viscosity (set at $\nu_f = 1.3 \times 10^{-6}$).

- **Model 2:** Almedeij (2008):

$$C_d = \left(\frac{1}{(\phi_1 + \phi_2)^{-1} + \phi_3^{-1}} + \phi_4 \right)^{1/10}$$

$$\phi_1 = (24 Re_p^{-1})^{10} + (21 Re_p^{-0.67})^{10} + (4 Re_p^{-0.33})^{10} + 0.4^{10}$$

$$\phi_2 = \frac{1}{(0.148 Re_p^{0.11})^{-10} + 0.5^{-10}}$$

$$\phi_3 = (1.57 \times 10^8 Re_p^{-1.625})^{10}$$

$$\phi_4 = \frac{1}{(6 \times 10^{-17} Re_p^{2.63})^{-10} + 0.2^{-10}}$$

Note

The drag model can be user-defined by using a custom `LagrangianParticleSet` class (see the Particle Modules section).

4. Diffusion

4.1. Diffusion option (set via `param:diffusion_lsm2_option`)

- **Option 1:** diffusion only on $\mathbf{x}_p$ with K_i given ($B_{p,i} = 0$);

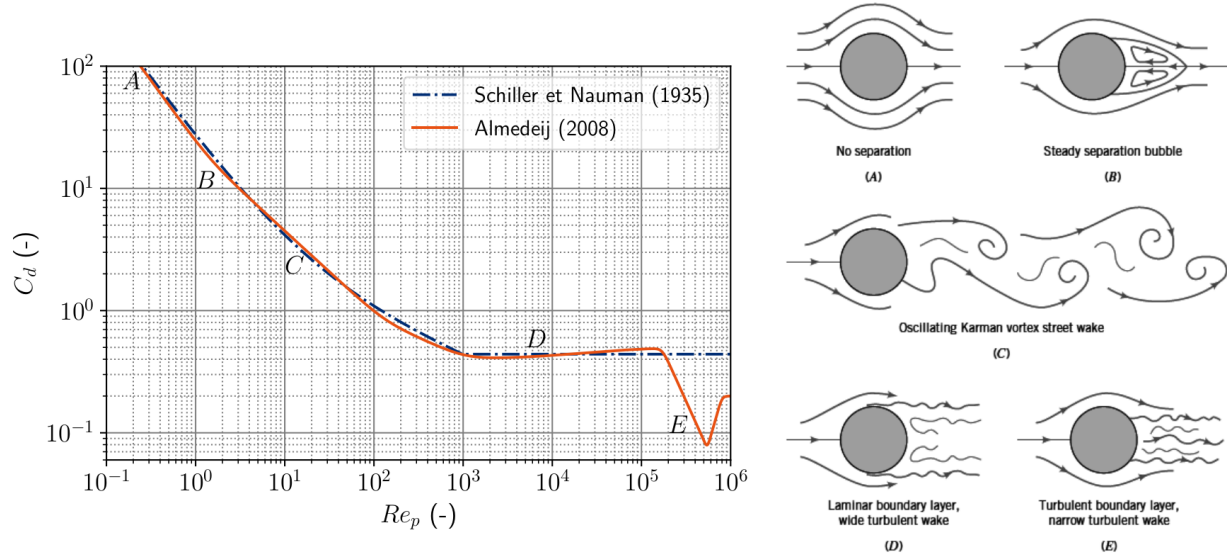


Fig. 10: Figure 6: Drag coefficient of a sphere as a function of the Reynolds number (left) and illustration of the flow pattern around the particle depending on the Reynolds number (right).

- **Option 2:** diffusion only on $\mathbf{U}_p$ with $B_{p,i}$ given ($K_i = 0$);
- **Option 3:** diffusion only on $\mathbf{U}_p$ with K_i given and $B_{p,i} = \mathcal{A}_1 \sqrt{2K_i}$.

Note

Adding stochastic terms to both position and velocity at the same time is not possible in CyLag.

4.2. Diffusion models (set via param:diffusion_model)

- **Model 0:** no diffusion, $K_i = 0$ or $B_{p,i} = 0$, ($0 \leq i \leq 3$).
- **Model 1:** constant isotropic diffusivity with user-defined K_i or $B_{p,i}$ (see param:diffusion_lsm2_option), set via param:horizontal_diffusivity and param:vertical_diffusivity, similarly to the LSM-1.
- **Model 2:** $k - \varepsilon$ turbulence model for the horizontal component and constant vertical diffusivity. The horizontal diffusivity is then defined as follows:
 - if param:diffusion_lsm2_option=1: $K_i = \nu_t / \sigma_c$ ($0 \leq i \leq 2$), in which σ_c is the Schmidt number (param:schmidt_number, default : 0.72) and ν_t is the turbulent diffusivity defined as $\nu_t = C_\mu k^2 / \varepsilon$, where C_μ is the Prandtl-Kolmogorov constant equal to $C_\mu = 0.09$;
 - if param:diffusion_lsm2_option=2 or param:diffusion_lsm2_option=3: $B_{p,i} = \mathcal{A}_1 T_L \sqrt{C_0 \varepsilon}$, ($0 \leq i \leq 2$), in which case T_L is the Lagrangian time scale, defined as $T_L = \frac{k}{\varepsilon} / (\frac{1}{2} + \frac{3}{4}C_0)$, where C_0 is the Kolmogorov constant (equal to 1.2).
- **Model 3:** $k - \varepsilon$ turbulence model for both horizontal and vertical components.

Note

When using the $k - \varepsilon$ turbulence model, the mean Eulerian fields k and ε should be passed to the EulerianFieldSet object with the optional_fields parameter.

5. Time schemes (set via param:time_scheme)

- param:time_scheme=1: **Euler** (1st order);
- param:time_scheme=2: **Runge-Kutta 2** (2nd order);
- param:time_scheme=3: **Runge-Kutta 3** (3rd order);
- param:time_scheme=4: **Runge-Kutta 4** (4th order);
- param:time_scheme=5: **Direct Integrator** (1st order, default) analytical solution over one time step, unconditionally stable.

Warning

Explicit schemes (**Euler** and **Runge-Kutta**) have a relaxation stability restriction. For a scalar frozen drag rate, **Euler** is linearly stable for $0 < dt < 2\tau_p$; $dt \leq \tau_p$ additionally avoids oscillatory relaxation. The **Direct Integrator** removes this linear stability restriction, but does not remove accuracy requirements associated with varying drag, Eulerian fields, additional forces or boundary crossings. It is not an exact solution of the nonlinear variable-coefficient problem.

Warning

High order anticipating methods such as the **Runge-Kutta** schemes are only suitable for ODE systems, ensure the diffusion is negligible before using these schemes.

2.4 Lagrangian Stochastic Model n°3 (LSM-3)

The LSM-3 formulation represents the most complete model available in CyLag. In addition to particle position and velocity, the state vector includes the fluid velocity seen by the particle. The evolution of this fluid velocity is described through a Langevin stochastic equation, allowing the model to explicitly represent turbulent velocity fluctuations and their temporal correlations. LSM-3 therefore belongs to the family of Generalized Lagrangian Stochastic Models (GLSMs) and is particularly appropriate when the interaction between particle dynamics and turbulence must be resolved with greater physical realism. Although computationally more demanding, it provides the most comprehensive description of particle transport and dispersion processes.

1. Particle state vector: $\{\mathbf{x}_p, \mathbf{U}_p, \mathbf{U}_s\}$ (position, velocity, and fluid velocity seen)

2. Main equations:

$$\begin{aligned} dx_{p,i} &= U_{p,i} dt, \\ dU_{p,i} &= \mathcal{A}_0 g_i dt + \mathcal{A}_1 (U_{s,i} - U_{p,i}) dt + \mathcal{A}_2 dU_{s,i} + \mathcal{A}_3 F_{a,i} dt, \\ dU_{s,i} &= -\frac{1}{\rho_f} \partial_i P_f dt - \frac{1}{T_{L,i}} (U_{s,i} - U_{f,i}) dt + B_{s,i} dW_i. \end{aligned}$$

where:

- $i \in \{1, 2, 3\}$ indicates the axis x, y, z respectively,
- $\mathbf{x}_p$ is the particle's position,
- $\mathbf{U}_p$ is the particle's velocity,
- $\mathbf{U}_s$ is the fluid velocity seen by the particle,
- $\mathbf{F}_a$ is an additional force (user-defined or module-dependent),

- $\mathbf{P}_f$ is the mean pressure field of the fluid at the particle's position,
- The coefficients $\mathcal{A}_i$ are defined as for LSM-2 by replacing $\mathbf{U}_f$ by $\mathbf{U}_s$ in $\mathcal{A}_1$ and Re_p ,
- $T_{L,i}$ are the Lagrangian time scales,
- $B_{s,i}$ are the stochastic diffusion coefficients,
- $\mathbf{W}$ is a Wiener process, whose increment $\mathbf{W} = \mathbf{W}(t + dt) - \mathbf{W}(t)$ is a Gaussian variable with zero mean and variance dt ,

The Lagrangian time scale and stochastic diffusion coefficient are usually defined from turbulent mean fields. When using the $k - \varepsilon$ model, the following definitions are used, for $i \in \{1, 2, 3\}$:

$$T_{L,i} = \frac{k}{\varepsilon} / \left(\frac{1}{2} + \frac{3}{4} C_0 \right) \quad \text{and} \quad B_{s,i} = \sqrt{C_0 \varepsilon}$$

where:

- C_0 is the Kolmogorov constant (equal to 1.2),
- k is the turbulence kinetic energy,
- ε is the rate of dissipation of turbulence kinetic energy,

Since the Eulerian pressure field is usually unavailable in free surface flow solvers, its contribution to the Langevin drift is not evaluated explicitly. Instead, it is represented by the resolved variation of the mean fluid velocity along the particle trajectory, evaluated from successive interpolated Eulerian velocities. The following approximation is used:

$$-\frac{1}{\rho_f} \partial_i P_f dt \simeq dU_{f,i}(t, \mathbf{x}_p(t)) .$$

3. Drag models: the same options as for LSM-2 are available with LSM-3 which we recall here. The drag coefficient C_d can be set via the parameter `pset:drag_coefficient_model`:

- **Model 0:** constant coefficient defined via the `pset:drag_coefficient` parameter,
- **Model 1:** Schiller (1933),
- **Model 2:** Almedeij (2008).

4. Diffusion models (set via `param:diffusion_model`)

Diffusion with the LSM-3, meaning the couple $(T_{L,i} - B_{s,i})$, is usually provided by the couple of mean fields $(k - \varepsilon)$ interpolated in the particle's position. However, in environmental applications, the vertical and horizontal turbulence are often modelled with different assumptions. Also, in pseudo-3D simulations, only horizontal $(k - \varepsilon)$ fields are known and users might want to apply a constant vertical diffusivity that is not computed from the horizontal $(k - \varepsilon)$ fields. It is therefore allowed to provide a couple $(T_{L,i} - K_i)$, in which case $B_{s,i}$ is defined as follows:

$$B_{s,i} = \sqrt{2K_i} / T_{L,i}$$

This option is available for both horizontal diffusion and vertical diffusion, depending on the value of `param:diffusion_model`:

- **Model 1:** constant isotropic diffusivity with user-defined $K_{s,i}$ set via `param:horizontal_diffusivity` and `param:vertical_diffusivity`, similarly to the LSM-1. the time scales $T_{L,i}$ are supposed constant and fixed by default to 1 second but can be adjusted via `param:diffusion_lsm3_tl_horizontal` and `param:diffusion_lsm3_tl_vertical`.
- **Model 2:** $k - \varepsilon$ turbulence model for the horizontal component and constant vertical diffusivity.
- **Model 3:** $k - \varepsilon$ turbulence model for both horizontal and vertical components.

Warning

The LSM-3 model cannot be used with `param:diffusion_model=0` (null diffusivity). Use LSM-2 without diffusion instead.

Note

When using the LSM-3 model the use of $k - \varepsilon$ turbulence model is mandatory and the mean Eulerian fields k and ε should be passed to the `EulerianFieldSet` object with the `optional_fields` parameter.

5. Time schemes (set via `param:time_scheme`)

- `param:time_scheme=1`: **Euler-Maruyama** (1st order);
- `param:time_scheme=5`: **Direct integrator with diffusion** (1st order, default).

Warning

Euler-Maruyama requires resolution of both relaxation scales, the sufficient non-oscillatory bound is $dt < \min(\tau_p, T_L)$ with $\tau_p = 1/\mathcal{A}_1$. Exact frozen-coefficient integration (**Direct integrator**) removes the linear relaxation stability restriction, not the need to resolve changes in the model coefficients. For both direct integrators, Gaussian RNG methods 2 or 3 are required for an exact Gaussian transition. Method 1 preserves the prescribed first two moments of each frozen step but uses rectangular, non-Gaussian increments.

2.5 Pseudo-3D velocity corrections

In the pseudo-3D case we introduce correction terms to account for the vertical variation of the horizontal velocity magnitude. To do so, we assume a logarithmic velocity profile in the vertical direction, and we apply a correction to the horizontal velocity components of the particles based on their vertical position. The corrected horizontal velocity $\mathbf{U}_c \in \mathbb{R}^2$ is computed as follows:

$$\mathbf{U}_c(z) = \mathbf{U}_f \left[\ln \left(\frac{z - z_b}{z_0} \right) \right] / \left[\ln \left(\frac{h}{z_0} \right) - 1 \right]$$

where

$$z_0 = h \exp \left[- \left(1 + \frac{\kappa}{\sqrt{C_f/2}} \right) \right]$$

with κ the von Karman constant (set at $\kappa = 0.41$) and C_f the friction coefficient of the Eulerian field set simulation. In CyLag, we don't use C_f directly, but deduce it from the friction law (`fset:friction_model`) and friction coefficient (`fset:friction_coefficient`). The friction laws available in CyLag are as follows:

- **Model 0**: Chézy law (`fset:friction_coefficient` is the Chézy coefficient),
- **Model 1**: Strickler law, default (`fset:friction_coefficient` is the Strickler coefficient, default: 40.),
- **Model 2**: Manning law (`fset:friction_coefficient` is the Manning coefficient),
- **Model 3**: Nikuradse law (`fset:friction_coefficient` is the Nikuradse roughness height).

The vertical Eulerian velocity being set to zero in pseudo-three-dimensional mode, to avoid particle artificial vertical motion due to changes in free surface and bottom elevation along the particle path, we introduce an additional vertical

velocity. We define the relative height by $\sigma = (z - z_b)/(z_s - z_b)$, where z_b is the bed elevation and z_s is the free surface elevation. The additional vertical velocity is then defined as follows:

$$w_c(z) = (1 - \sigma) \frac{Dz_b}{Dt} + \sigma \frac{Dz_s}{Dt},$$

where $D/Dt = \partial/\partial t + \mathbf{U}_c \cdot \nabla$ is the material derivative along the pseudo-3D corrected horizontal velocity.

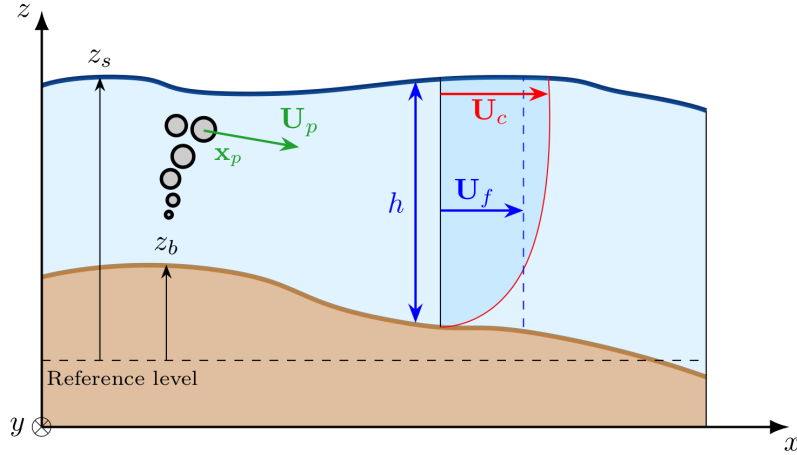


Fig. 11: Figure 6: Pseudo-3D illustration.

2.6 Particle Modules

In CyLag, users can define custom `LagrangianParticleSet` classes that inherit from the parent class. This allows the implementation of specific physics, such as custom drag-coefficient formulas, custom added drift velocity or force, and particle behavior models for aquatic wildlife, among others. You can build your own custom model in `cylag/modules/custom_particle_set.pyx`.

Several custom classes are already defined to resolve specific types of particles, such as:

- **Oil slick** (`cylag/modules/oil_slick.pyx`): simulates oil-slick transport and dispersion with the LSM-1 module;
- **Algae** (`cylag/modules/algae.pyx`): simulates the transport of macrophytes with LSM-1 or LSM-2 models;
- **Macrophytes** (`cylag/modules/macrophyte.pyx`): simulates the transport of macrophytes with LSM-1 or LSM-2 models;
- **Fish** (`cylag/modules/fish.pyx`): simulates the transport and behavior of fish with individual- or collective-based models;
- **Jellyfish** (`cylag/modules/jellyfish.pyx`): simulates the transport of jellyfish.

Each new type of `LagrangianParticleSet` has its own set of input parameters (see docstrings of each class).

PARTICLE-MESH IMPLEMENTATION

3.1 General overview of the algorithm

CyLag’s hybrid Eulerian-Lagrangian implementation relies on an Eulerian solver, whose results are computed on a 2D triangular mesh or a 3D mesh (triangular prismatic elements). CyLag’s Lagrangian solver then computes the motion of particles that are embedded in the computational domain using Lagrangian Stochastic Models (LSMs) described in section *Mathematical models*.

The time loop associated with CyLag’s Lagrangian solvers can be executed independently from the Eulerian solver in **offline** mode. The offline mode implies that only one-way coupling can be achieved. More importantly, it requires the Eulerian results to be saved with a sufficiently high frequency in order to avoid errors induced by poor time resolution. These drawbacks can be avoided by using the Lagrangian solver in the same time loop as the Eulerian solver, which is called **online** mode. This mode is available through the use of both CyLag and TELEMAT Python APIs.

Solving Lagrangian Stochastic Models (LSMs) relies on several successive steps, which are illustrated in Figure 1. Particles are first added within the domain, and then the mean fields (such as the fluid velocity) are interpolated at the location of the particles. The particle equations (LSMs) are solved using appropriate time schemes, and the particle state vector is updated. After the update, the particles are localized, and boundary conditions are applied if any boundary edge (2D) or surface (3D) has been crossed. Boundary conditions include rebounds on the walls or outlet boundary conditions. Finally, the particle state vectors are written to output files. In the following, the algorithms implemented for interpolation, particle search, and boundary conditions are described.

3.2 Interpolation

1. Time interpolation

When using CyLag in offline mode, the mean fields required for the Lagrangian solver are not available at the same times if the time step of the Lagrangian solver is different from the Eulerian solver. In this case, linear time interpolation is carried out to compute the mean fields at the Lagrangian time t^n from the closest Eulerian times t^k and t^{k+1} such that

$$\mathcal{F}(t^k \leq t^n < t^{k+1}, \mathbf{x}) = \alpha \mathcal{F}(t^k, \mathbf{x}) + (1 - \alpha) \mathcal{F}(t^{k+1}, \mathbf{x}),$$

where $\mathcal{F}(t^n, \mathbf{x})$ is the Eulerian mean field evaluated at Lagrangian time step t^n , and where $\alpha = \frac{t^{k+1} - t^n}{t^{k+1} - t^k}$.

2. Spatial interpolation

Eulerian mean fields interpolation at the particle position is done via linear interpolation by default as presented in Figure 3. In 3D, the results files are assumed to contain only triangular prismatic elements, which result from an extrusion of a 2D triangular mesh (TELEMAT-3D result files).

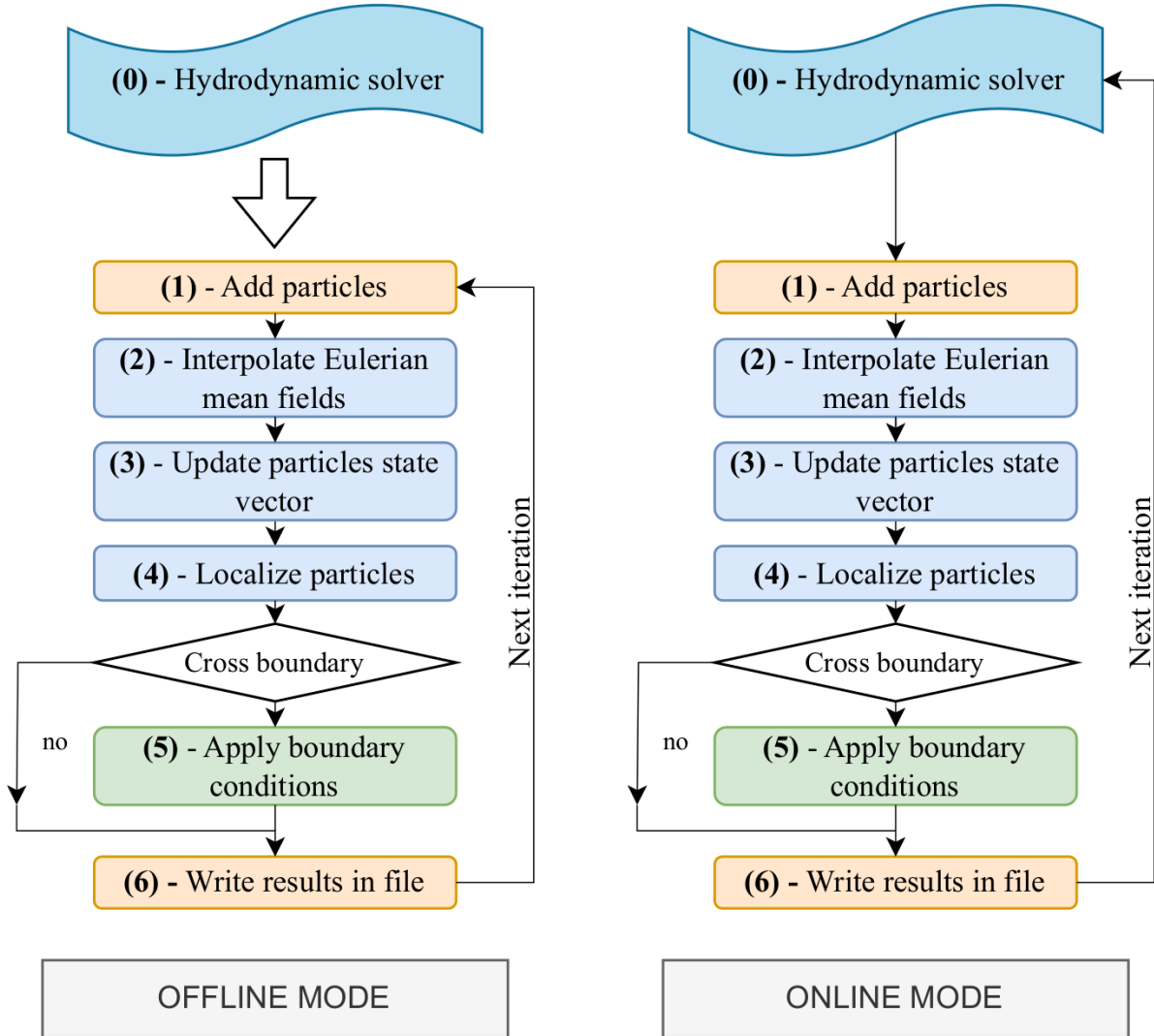


Fig. 1: Figure 1: Flowchart of the main time loop of CyLag with the OFFLINE mode (left) or ONLINE mode (right). Step (1) and (6) are optional and are executed only at chosen time steps.

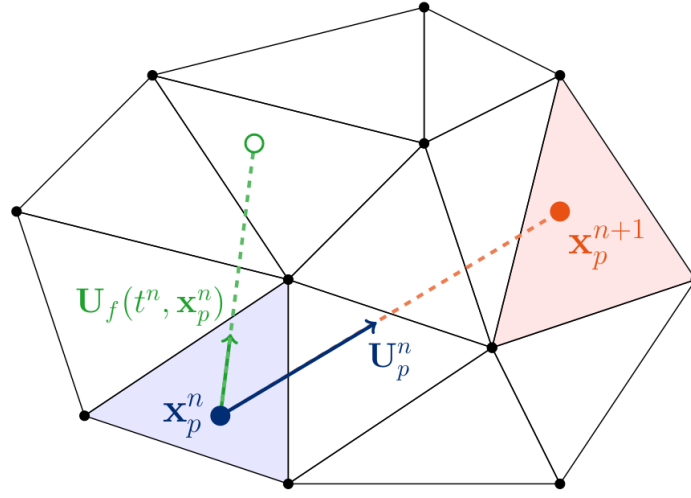


Fig. 2: Figure 2: Particle position update and trajectory separation illustration. The green line indicates the fluid material path line whereas the red line indicates the particle trajectory.

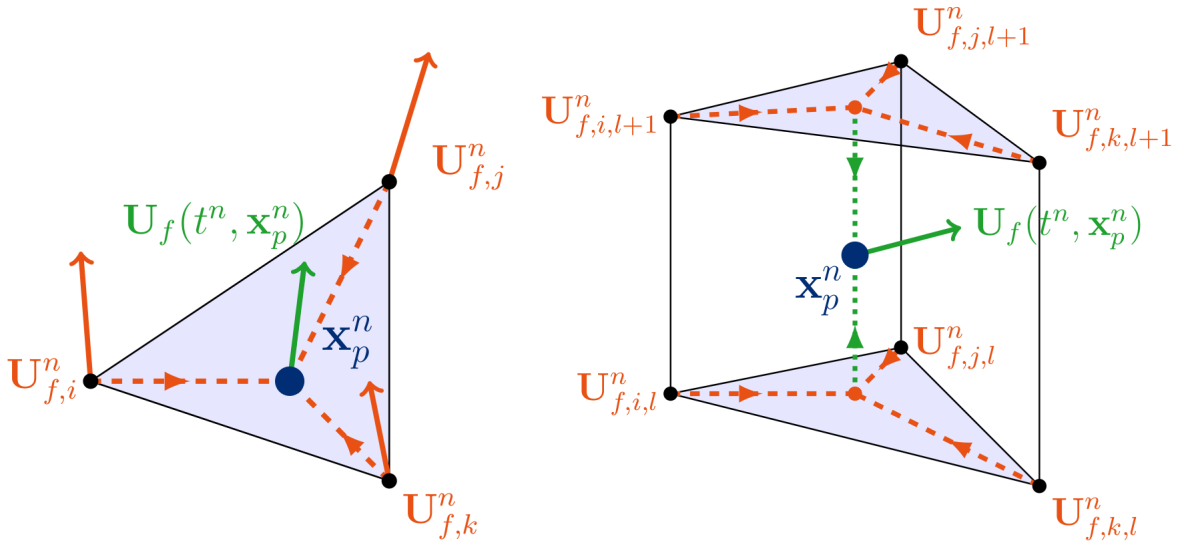


Fig. 3: Figure 3: Interpolation of the fluid mean velocity field at the particle's location in 2D (left) and 3D (right). Indices i, k, j are the nodes of the 2D triangular mesh while l and $l+1$ are the indices of the 3D mesh layers.

3.3 Search algorithm

Once particle positions are updated, their location in the mesh (triangle and surrounding layers) needs to be updated in order to interpolate at the next time step and to apply boundary conditions if the particles are crossing boundary edges (2D) or surfaces (3D).

1. Horizontal search

Several search algorithms have been implemented in CyLag:

- **KDTree:** Search all triangles (using a KDTree)
 - Used for initialization
 - $O(\log n + k)$ complexity
- **Neighborhood Search:** Search neighboring triangles
 - Uses triangle adjacency tables
 - $O(1)$ if particle hasn't moved far
- **Path Following:** Trace a segment from the old to the new position
 - Finds intersections with mesh edges
 - Efficient for moderate displacements
 - Returns boundary edge if particle exits
- **Hierarchical:** Path $\rightarrow$ KDTree

By default, the hierarchical search is used in CyLag. First, we carry out a Path Following search, and if the particle is not found, we use a KDTree search as a fallback.

The recursive edge-intersection and edge-to-triangle mapping algorithm (see Figure 4) consists of starting from the previous known position $\mathbf{x}_p^n$ and the associated triangle T_i . The edge of the triangle T_i that is intersected by the segment $[\mathbf{x}_p^n, \mathbf{x}_p^{n+1}]$ is determined in order to search in the neighboring triangle T_j . If the position $\mathbf{x}_p^{n+1}$ is not found in T_j , we start again from T_j and eliminate the previous intersected edge from the intersection search. If the point $\mathbf{x}_p^{n+1}$ lies in the neighboring triangles of T_i , the maximum cost of the algorithm is only three intersections and one point-in-triangle test. Note that the optimized search algorithm requires the previous position to be known; the particles are therefore localized using KDTree search for initialization.

2. Vertical search (for 3D only)

For 3D applications, horizontal localization is unchanged, while vertical localization is achieved by interpolation of the free surface and bottom at the particle location and assuming a regular vertical layer spacing. The algorithm could be adapted to irregular vertical discretization as long as an analytical formula can be provided for the layer spacing and if the layer spacing does not vary in space.

3.4 Boundary conditions

Different boundary conditions are applied depending on the type of boundary edge (or surface) that a particle goes through. We distinguish three main types of boundary conditions.

- **Open inlet boundary:** particles are entering the domain through an open inflow boundary condition.
- **Open outlet boundary:** particles are leaving the domain and are removed from the pool of active particles.
- **Wall boundary (rebound):** particles interact with a wall, which can be a lateral wall, the bottom, or the free surface.

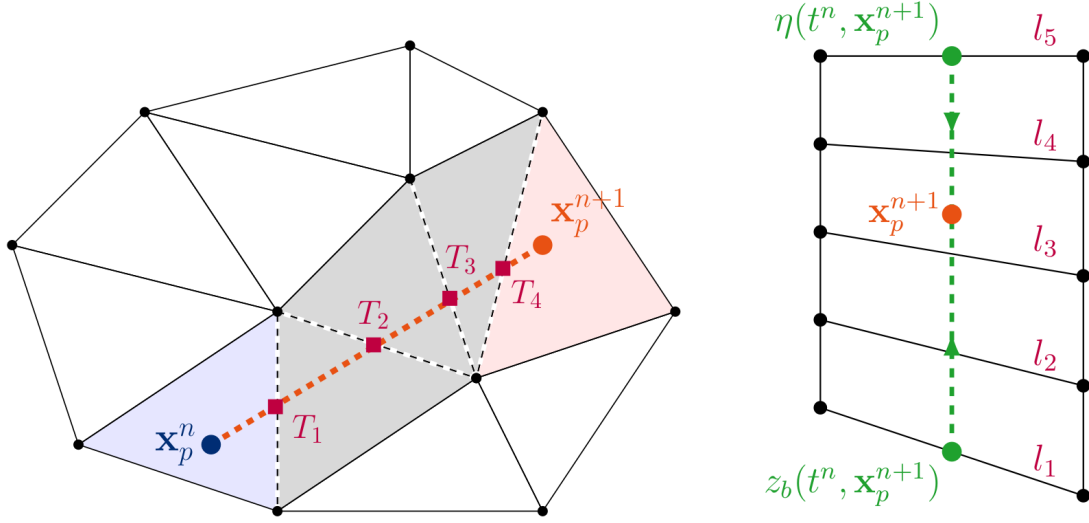


Fig. 4: Figure 4: Illustration of the 2D optimized search algorithm (left) and vertical localization algorithm (right). Purple rectangles indicate successive intersections and T_j the triangles in which the new particle position is searched. In this example, four points in triangle tests are performed until the position is found.

1. Rebound boundary conditions

Wall boundary conditions are very important in order for the particles to stay inside the computational domain and maintain the well-mixed condition. Two types of rebound can be considered, as illustrated in Figure 5. The most straightforward is the specular rebound, which corresponds to the rebound of a spherical particle on a planar surface. The second option is the diffuse rebound, for which the direction of the particle velocity vector after the rebound is no longer deterministic but rather follows a PDF that describes the dispersion of the trajectories due to surface roughness. The choice between specular and diffuse rebound is made through the parameter `param:boundary_conditions_type` (1 for specular, 2 for diffuse).

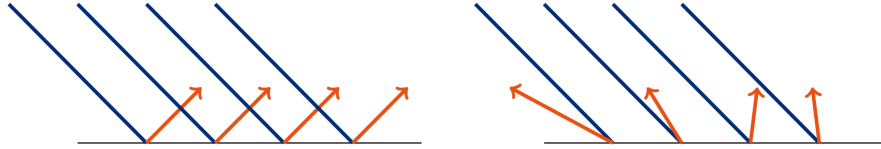


Fig. 5: Figure 5: Specular rebound (left) and diffuse rebound (right).

1.1 Specular rebound

The computation of specular rebounds is based on intersections and symmetric point computations. In 2D, it only involves segment intersection and reflection, while in 3D, it requires segment-face intersection and surface reflection, which are computationally more demanding. The rebound algorithm implemented in CyLag is compatible with multiple reflections to avoid stuck particles in complex geometries. An example is given in Figure 6 for 2D planar reflections. A maximum number of rebounds is allowed in order to avoid excessive cost due to stuck particles (`param:max_wall_rebounds`, set to 4 by default). Stuck particles can occur in very shallow 3D models when wetting and drying are present in the model. Also, another simplified algorithm is proposed for such cases, in which only 2D reflections are computed on the side of the 3D mesh. Besides, a damping coefficient is added to model the energy loss due to the collision (`param:rebound_damping_coef`), in which case the velocity of the particle is reduced after the rebound. The default damping is set to 0.5, i.e. particles lose half of their kinetic energy in collisions. Additionally, when particles reach the free surface or the bottom, the reflection distance is limited to half the water depth to avoid a very high number of reflections in drying areas where the water depth is close to zero.

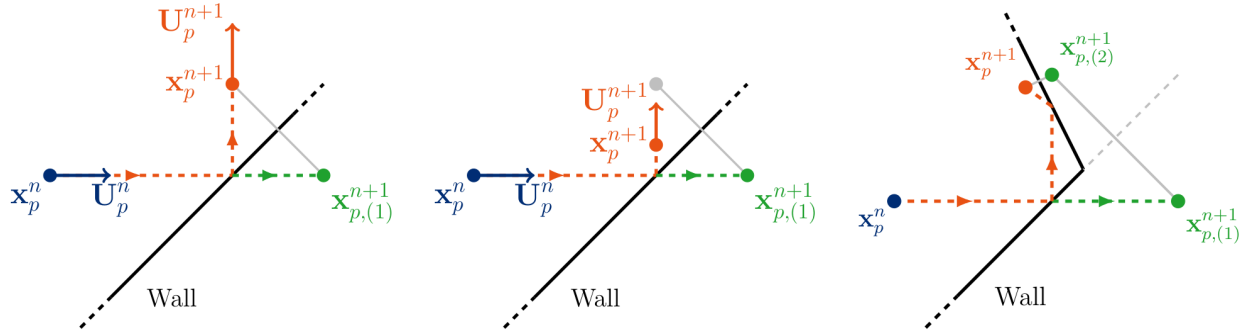


Fig. 6: Figure 6: 2D illustration of specular rebound boundary condition without damping (left), with damping (center) and example of multiple rebounds (right). The red dotted line represents the real trajectory of the particle after rebound while the green dotted line represents the trajectory without rebound.

1.2 Diffuse rebound

Following the classic stochastic rough-wall approach (Sommerfeld, 1992), the true geometric wall is kept for repositioning the particle, but the local wall normal used for the velocity reflection is randomly perturbed by an angle θ sampled around 0 with std-dev `param:wall_roughness_angle` (in degree), drawn via the RNG function. This simulates microscopic surface roughness causing quasi-random scattering instead of a perfect mirror bounce, while still conserving speed/energy consistently with the existing damping scheme.

2. Stranding

When particles reach a wall boundary that corresponds to a coastline, the particles could stay on the shore. To take that into account, a stranding probability is included in CyLag (see `param:stranding_probability`). When a particle rebound is triggered, the stranding probability is applied; if stranded, the particle is deleted from the active pool of particles.

SETUP, INPUTS AND PRACTICAL INFORMATIONS

4.1 Setup philosophy

CyLag is run using its Python API; to launch a simulation, one has to define a Python script in which cylag is imported and cylag objects are defined. Each object has its own inputs and parameters:

1. `EulerianFieldSet`: requires either a 2D or 3D hydrodynamic result file or NumPy arrays containing the Eulerian fields,
2. `LagrangianParticleSet`: requires parameters related to the physical properties and initialization of particles,
3. `Solver`: takes `EulerianFieldSet` and `LagrangianParticleSet` as inputs, as well as a `parameters` dictionary containing the main solver options.

The simulation script usually looks like this:

```
import cylag

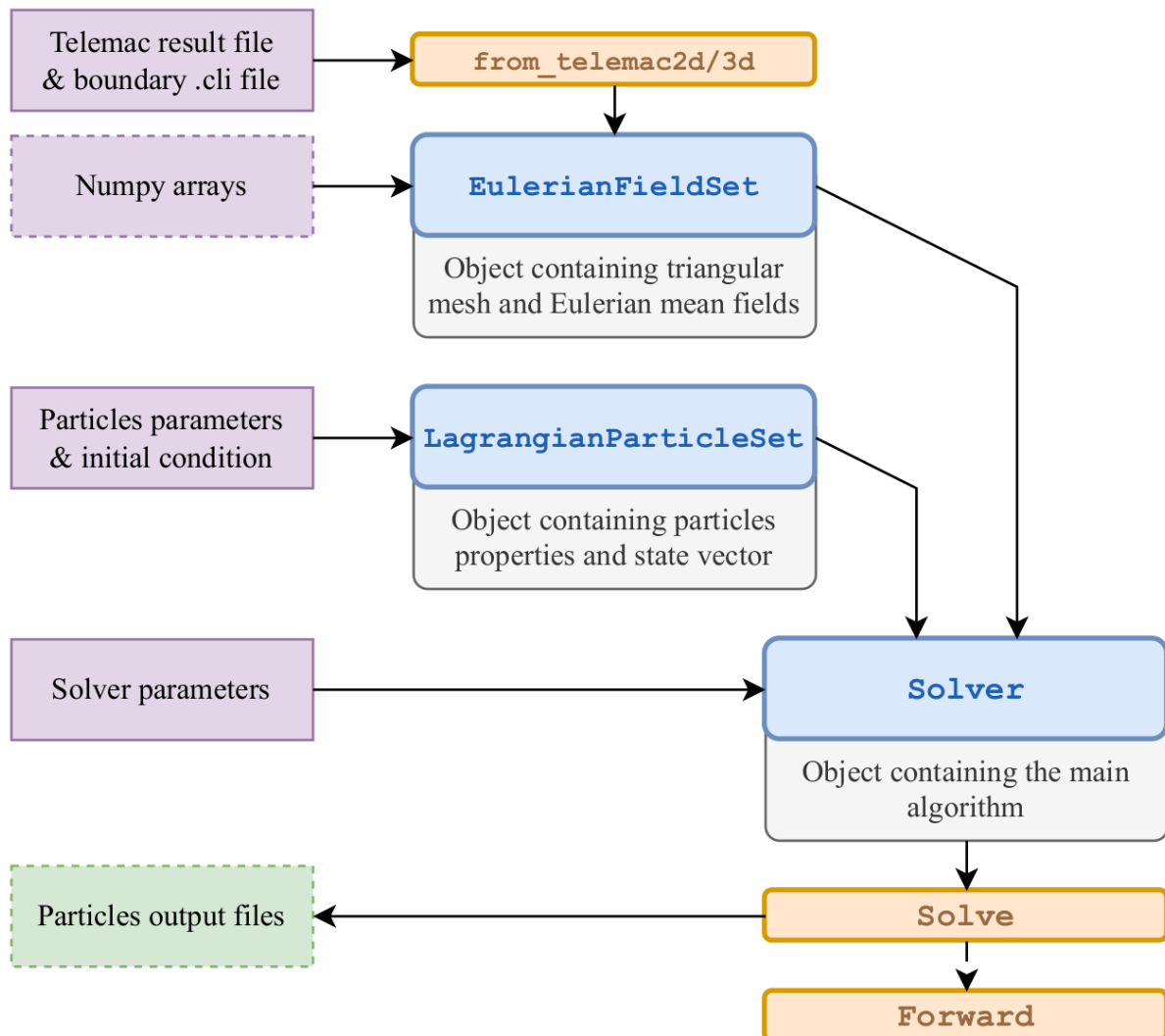
# 1. define EulerianFieldSet from TELEMAC-2D results:
fset = cylag.EulerianFieldSet.from_telemac2d(
    'telemac_result_2d.slf',
    bnd_file='telemac_geo.cli')

# 2. define LagrangianParticleSet:
ini_position = np.asarray([
    [5000., 0.],
    [5000., 500.],
    [1000., 0.],
    [1000., 500.]])

pset = cylag.LagrangianParticleSet(
    position=ini_position,
    dim=2,
    initial_pool_size=4)

# 3. define the solver:
parameters = {
    'final_time': 43200.,
    'time_step': 1.,
    'model': 1,
    'time_scheme': 1,
    'listing': True,
```

(continues on next page)



(continued from previous page)

```

'listing_printout_period': 100,
'output_file': True,
'output_printout_period': 10,
'output_file_format': 'txt',
'output_file_name': 'particles_2d',
}

cylag_solver = cylag.Solver(fset, pset, parameters)

# 4. run CyLag
cylag_solver.solve()

```

Note

The distinction between 2D and 3D is made based on the `dim` parameter of the `EulerianFieldSet` and `LagrangianParticleSet` objects.

4.2 Overview of input parameters

In this section we provide a simple description of the main input parameters for each class. Users can also refer to CyLag's API for a more complete description of inputs (types, array size, default values, etc.).

4.2.1 1 - EulerianFieldSet inputs

There are two ways to define the `EulerianFieldSet` object. The first one is to use the built in methods, in which case input parameters are automatically filled with imported values:

- `EulerianFieldSet.from_teleMAC2d()`,
- `EulerianFieldSet.from_teleMAC3d()` or
- `EulerianFieldSet.from_hdf5()`.

When initialize from TELEMAC, the main informations to provide are a result file (in `.slf` format) and a boundary condition file (in `.cli` format). Also optional fields can be added with a list name provided that they are save in the corresponding result file. An example is given here after:

```

file_name = os.path.join('data', 'r2d_meander.slf')
bnd_file = os.path.join('data', 'geo_meander.cli')

fset = cylag.EulerianFieldSet.from_teleMAC2d(
    file_name,
    bnd_file=bnd_file,
    optional_fields=['TURBULENT ENERG.', 'DISSIPATION'])

```

Note

Make sure the optional fields listed are saved in the `.slf` result file.

The second option is to pass inputs directly to `EulerianFieldSet`, which are as follows:

- `triangular_mesh` : Triangular mesh containing geometry and connectivity
- `dim` : Dimension of the mesh and fields (2 or 3, default: 2)
- `nlayers` : Number of layers for 3D mesh (default: 1)
- `times` : Times at which fields were recorded
- `elevation` : Elevation field for each point of the mesh
- `velocity_x` : X-coordinate of the velocity field
- `velocity_y` : Y-coordinate of the velocity field
- `velocity_z` : Z-coordinate of the velocity field (3D only)
- `opt_fields` : Optional fields (default: None)
- `opt_fields_names` : Names of the optional fields (default: None)

4.2.2 2 - LagrangianParticleSet inputs

- `position` : initial particle coordinates x,y,z (default: None)
- `velocity` : initial particle velocity up,vp,wp (default: None)
- `fluid_velocity_seen` : initial fluid velocity seen us,vs,ws (default: None)
- `dim` : dimension 2 or 3 (default: 2)
- `initial_pool_size` : initial size of particle pool (default: `size(position)`)
- `min_pool_size_update` : minimum size increment for pool expansion (default: 1000)
- `particle_density` : particle density (default: 1000.)
- `particle_diameter` : mean particle diameter (default: 0.01)
- `particle_diameter_distribution` : particle diameter distribution (default: None)
 - None: constant diameter, monodisperse
 - ['uniform', delta]: $U(\text{particle_diameter}-\text{delta}, \text{particle_diameter}+\text{delta})$
 - ['normal', std]: $N(\text{particle_diameter}, \text{std})$
 - ['lognormal', gsd]: $LN(\text{particle_diameter}, \text{gsd})$
 - ['weibull', shape]: $\text{scale}=\text{particle_diameter}/\text{tgamma}(1 + 1/\text{shape})$
- `drag_coefficient_model` : model for the drag coefficient, LSM-2 only (default: 1)
 - 0: Constant drag model,
 - 1: Schiller et Nauman (1935),
 - 2: Almedeij (2008).
- `drag_coefficient` : constant drag coefficient for `drag_coefficient_model=0`, LSM-2 only (default: 0.44)
- `added_mass_force` : activate added mass force, LSM-2 only (default: False)
- `added_mass_coef` : added mass coefficient, LSM-2 only (default: 0.5)
- `buoyancy_velocity_model`: add settling velocity, LSM-1 and `pset.dim=3` only (default: 0)
 - 0 : Constant buoyancy velocity;

- 1 : Analytical solution.
- `buoyancy_velocity`: settling velocity in m/s, LSM-1 only (default: 0.0)
- `compute_depth`: compute particle depth to free surface, only for `pset.dim=3` (default: False)
- `max_stranded`: maximum number of stranded particles (default: `initial_pool_size`)

Note

- When `position` argument is given, the `initial_pool_size` is automatically set to the size of `position` unless `initial_pool_size` is provided by the user.
- If `position` is not given and sources are used to add particles during the simulation, it is important to initialize `initial_pool_size` with a rough estimate of the number of particles that will enter the domain with sources. The `min_pool_size_update` corresponds to the size of the increase in pool size if the number of particles reach the `initial_pool_size` value during the computation. In that case, CyLag reallocates all particle attributes to match the new number of particles. This allocation change impacts performance, so it is recommended to fix a value that is not too low for `min_pool_size_update`.

4.2.3 3 - Solver inputs

There are two types of solvers in CyLag:

- `Solver`: for sequential usage,
- `SolverParallel` for parallelization with MPI (particle are distributed among processes).

Both classes take the following arguments:

- `field_set`: Eulerian field set containing velocity and optional fields
- `particle_set`: Lagrangian particle set
- `parameters`: Model parameters (default: defined in `cylag_dico.py`)
- `sources`: List of particle sources (default: None)
- `control_sections`: List of control sections for statistics (default: None)

4.2.4 4 - Parameters dictionary

Complete list of available parameters in `cylag_dico.py`:

Time Parameters

- `initial_time`: simulation start time (default: 0.0)
- `final_time`: simulation end time (default: 3600.0)
- `time_step`: time step size (default: 1.)
- `frozen_eulerian_fields`: use first time step of EulerianFieldSet for steady-state flow (default: False)

LSM Parameters

- `model`: Lagrangian Stochastic Model (default: 1)
 - 0 : testing kernel;
 - 1 : LSM1;

- 2 : LSM2;
 - 3 : LSM3.
- `time_scheme`: integration scheme (default: depends on model)
 - 1 : Euler or Euler-Maruyama (default for LSM-1 with diffusion);
 - 2 : Runge-Kutta 2;
 - 4 : Runge-Kutta 4 (default for LSM-1 without diffusion);
 - 5 : Direct integration (default for LSM-2 and 3);
- `particle_velocity_init`: type of initialization for Up (default: 1)
 - 0 : user defined (zero if not specified),
 - 1 : interpolated from mean fluid velocity;
- `rng_method`: random number generator method (default: 2)
 - 1 : rectangular approximation;
 - 2 : Ziggurat method (default);
 - 3 : Marsaglia's polar method.
- `lsm3_edge_opt`: edge-case treatment of the LSM-3 direct integrator (`time_scheme=5`) (default: 2)
 - 1 : normalized Taylor series (accurate for tiny steps and $\mathcal{A}_1 = 1/T_L$);
 - 2 : closed-form coefficients with clipping: the degenerate denominator $\mathcal{A}_1 - 1/T_L$ is clipped, and the covariance is clipped to remain positive semi-definite instead of raising an error.

Physical Parameters

- `water_density`: density of water (default: 1000.)
- `diffusion_model`: main diffusion model (default: 0)
 - 0 : no diffusion,
 - 1 : constant diffusion coefficient,
 - 2 : diffusion coefficient from $k - \varepsilon$
- `horizontal_diffusivity`: horizontal diffusivity in m^2/s for `diffusion_model=1` (default: 0.0)
- `vertical_diffusivity`: vertical diffusivity in m^2/s (default: 0.0)
- `schmidt_number`: Schmidt number for turbulent `diffusion_model=2` and 3 (default: 1.0)
- `diffusion_lsm2_option`: option for diffusion coefficient in LSM-2 (default: 1)
 - 0 : B_i provided;
 - 1 : K_i provided.

Boundary Conditions

- `boundary_conditions`: enable BC handling (default: True)
- `boundary_conditions_type`: type of BC handling (default: 0)
 - 1 : specular rebound;
 - 2 : diffuse rebound.
- `boundary_conditions_debug`: verbose BC bounces for debug (default: False)

- `max_wall_rebounds`: maximum wall bounces (default: 4)
- `rebound_damping_coef`: damping on rebound (default: 0.5)
 - 0.0 : no kinetic energy loss;
 - 1.0 : total kinetic energy loss.
- `wall_roughness_angle`: standard deviation (in degree) of the random perturbation applied to the local wall normal to model diffuse rebounds (default: 15.0). low value corresponds to small roughness. Maximum value is 85 degrees.
- `stranding_probability`: stranding probability (default: 0.)
 - 0.0 : no stranding;
 - 1.0 : always stranding.

I/O Parameters

- `listing`: print progress (default: False)
- `listing_printout_period`: iterations between prints (default: 10)
- `output_rep`: name of the output repertory (default: “particles”)
- `output_file`: write particle positions (default: False)
- `output_printout_period`: iterations between outputs (default: 10)
- `output_file_format`: “vtk”, “txt” or “all” (“all” save both file format, default: “txt”)
- `output_file_name`: name of the output file (default: “particles”)
- `bnd_statistics`: track boundary crossings (default: False)
- `bnd_statistics_file_name`: base name of the boundary statistics files (default: “bnd_stats”),
- `control_sections_file_name`: base name of the control sections files (default: “cs_stats”),
- `stranding_output`: activate output file for stranded particles (default: False),
- `stranding_output_file_name`: name of the output file for stranded particle (default: “stranded_particles”),

Note

See the *Good practices* for recommendations on how to set main input parameters for a simulation.

4.3 Adding particles to a simulation

Adding particles to a simulation can be done in different ways:

- Add directly in initialization of `LagrangianParticleSet`;
- Add via `ParticleSource`;
- Add via `ParticleSourceFromField`.

Note that both `ParticleSource` and `ParticleSourceFromField` use a time `Scheduler` object to trigger the addition of particles in the domain. The time scheduler can be defined as follows:

```
source_scheduler = cylag.schedules(time_delta=5.)
```

Several options are available to schedule the addition of particles:

- `ini_time` (float) : Time at which the scheduler is initialised.
- `time_delta` (float) : Time delta between two tasks.
- `iteration_delta` (int) : Number of iterations between two tasks.
- `times` (list (float)) : Task times.
- `iterations` (list (int)) : Task time iterations.
- `count` (int) : Number of tasks, equally time-separated.
- `always` (bool) : Task is triggered at each time iteration.

4.3.1 1 - Add as input of `LagrangianParticleSet`

The easiest way to define particles is to add them at the beginning of a simulation via the `position` argument of the `LagrangianParticleSet` class, such as:

```
ini_position = np.asarray([
    [5000., 0.],
    [5000., 500.],
    [1000., 0.],
    [1000., 500.]])

pset = cylag.LagrangianParticleSet(
    position=ini_position,
    dim=2,
    initial_pool_size=4)
```

Users can also define the initial particle velocity and the fluid velocity seen (for LSM-2 and LSM-3) via the `velocity` and `fluid_velocity_seen` parameters. These velocities can also be initialized from the interpolation of the mean fluid velocity if `param:particle_velocity_init=1`. Note that if `particle_velocity_init=0` and the `velocity` parameter is not specified, it is initialized to zero.

Note

It is not recommended to set initial particle velocity to zero with LSM-2 and LSM-3.

4.3.2 2 - Add via `ParticleSource`

When using the `ParticleSource` class, users need to specify a polygon in which to add particles, as well as a time scheduler:

```
source_poly = np.array([[5.1, 0.5], [6.1, 0.5], [6.1, 1.5], [5.1, 1.5]])
source_scheduler = cylag.schedules(time_delta=5.)
source = cylag.ParticleSource(
    source_scheduler,
    source_poly,
    xy_method=1,
    xy_npart=4)
```

The main parameters of the `ParticleSource` are as follows:

- **scheduler**: class used to trigger the addition of particles
- **poly**: contains the x and y positions of the polygon shape
- **xy_method**: method of initialization on the horizontal plane (default: 0)
 - 0 : number of particles along x and y axis;
 - 1 : total number of particles;
 - 2 : number of particles/m²)
- **grid_res**: number of points on the x, y, and z axes
- **xy_npart**: number of particles in the polygon
- **xy_density**: number of particles per unit area in the polygon
- **z_method**: method of initialization on the vertical axis (default: 0)
 - 0 : number of particles along z axis;
 - 1 : number of particles layers;
 - 2: number of particles/m.
- **z_npart**: number of particles along the vertical axis
- **z_density**: number of particles per unit area along the vertical axis
- **zmin**: minimum z value for vertical initialization
- **zmax**: maximum z value for vertical initialization

4.3.3 3 - Add via ParticleSourceFromField

The idea of `ParticleSourceFromField` is to trigger the release of particles when a specific condition is met on a field of the `EulerianFieldSet` object. For example, particles can be added when a field reaches a given threshold. The Eulerian fields used are specified with their ID (the number in the list of optional fields); two fields are required:

- **field_id_criteria**: field used to compute the criterion;
- **field_id_density**: field used for density when adding particles.

When using the `ParticleSourceFromField` class, two options are available: **val** (value) and **var** (variation) for adding particles:

- **val**: the value of the field is compared to a **threshold**;
- **var**: the value of the field variation (`field[n+1]-field[n]`) is compared to a **threshold**.

The following comparison options are available (see the **criteria** parameter):

- **eq** (==);
- **neq** (!=);
- **geq** (>=);
- **leq** (<=);
- **gt** (>);
- **lt** (<).

Finally, users need to provide the **virtual_mass** associated with particles in kg to control the amount of particles added to the domain (default: 1.):

- If `virtual_mass == 1.:` `npart = density*cell_area;`
- If `virtual_mass != 1.:` `npart = density*cell_area/virtual_mass.`

An example is given below:

```
source = cylag.ParticleSourceFromField(
    scheduler=cylag.schedules(always=True),
    field_id_criteria=7,
    field_id_density=7,
    criteria='var_neq',
    threshold=0.,
    virtual_mass=20.,
    single_release=True)
```

4.4 Saving particles

Two output formats are available in CyLag:

- `.vtk` format: saves a list of files that can be loaded with ParaView;
- `.txt` format: saves a single `particles.txt` file that can be loaded with the `ParticlesIO` class of CyLag.

4.5 Post-processing particle files

The `ParticlesIO` class included in CyLag allows quick and easy post-processing of particle files. To load a particle file, simply use:

```
part = cylag.ParticlesIO.from_cylag_txt('particles/particles_2d.txt')
```

The positions can then be retrieved for any record (`rec`):

```
time = part.times[rec]
x = part.xp[rec][:]
y = part.yp[rec][:]
```

This class also allows simple computation of trajectories, based on all saved positions:

```
traj = part.get_trajectories(traj_list)
```

GOOD PRACTICES

5.1 Mesh and Eulerian fields quality

Accurate Lagrangian simulations require accurate hydrodynamic simulations as input.

- Avoid highly skewed triangles;
- Maintain uniform element size where possible;
- Verify your hydrodynamic simulation (it should be converged).

5.2 Number of particles

Lagrangian Stochastic Models are statistical models that converge by increasing the number of particles.

- Start with a low number of particles and gradually increase until convergence;
- Start with `initial_pool_size` > the maximum expected number of particles;
- Minimize pool resizing (because resizing the pool impacts performance).

5.3 Choice of numerical parameters

There are several numerical parameters that need to be adjusted depending on the type of simulation being carried out.

- Avoid huge time steps (there is no CFL restriction, but particles crossing a large number of cells at once impacts performance);
- Start with moderate time steps and reduce them until convergence;
- Gradually increase model complexity: start with LSM-1 and then switch to LSM-2 or LSM-3 if needed;
- Use the default schemes at the start: Euler for LSM-1 (scheme 1) and direct integration for LSM-2 (scheme 5);
- Test the different turbulence models and their impact on the results;
- Consider using high-order time integration methods if diffusion is negligible;
- Be mindful of time-step constraints for LSM-2 and LSM-3 (with Euler and RK schemes).

5.4 I/O frequency

Lagrangian-Eulerian simulations can be offline or online. The choice depends on the frequency of outputs for the Eulerian fields.

- Make sure the period of graphical printouts in your hydrodynamic simulation is sufficiently high (for offline mode);
- If the simulation time is too long (resulting in a large file size), consider switching to online mode;
- Balance data resolution and file size (avoid saving particle files at every time step);
- Use HDF5 for large datasets (compressed and fast).

DEVELOPER GUIDE

6.1 Get access

To get developer access to the repository, send an email to: fabien.souille@edf.fr

6.2 Development checklist

Before submitting any changes to CyLag, follow these simple steps:

- Create development branch and make your modifications;
- Compile the code in debug mode and run unit-tests to check non-regression:

```
python3 setup.py clean
python3 setup.py build_ext --inplace --debug
python3 -m pytest tests/*.py
```

- Compile the code in standard mode and run unit-tests to check non-regression:

```
python3 setup.py clean
python3 setup.py build_ext --inplace
python3 -m pytest tests/*.py
```

- Compile the Sphinx documentation and check the gallery of examples:

```
cd docs && make html
```

- Create a new example and documentation to validate your development;
- Create a merge request.

Note

Verify functionality, code quality, test coverage, documentation updates, dependency management, performance impact, and deployment readiness before submitting any changes.

6.3 Unit tests

After any modification of the source code and after recompiling, you should run unit tests to check non-regression. To run all unit-tests at once, type:

```
python3 -m pytest tests/*.py
```

You can also run unit tests of specific modules:

```
python3 -m pytest tests/test_00_geom_mesh.py
python3 -m pytest tests/test_14_core_solver.py
python3 -m pytest tests/test_20_lsm1_analytical_advection_2d.py
```

The test categories are organized as follows:

- **test_0X**: Geometry and mesh operations
- **test_1X**: Core classes
- **test_2X**: LSM-1 verifications
- **test_3X**: LSM-2 verifications
- **test_4X**: LSM-3 / RNG verifications
- **test_5X**: Extra modules
- **test_6X**: Parallel functionality
- **test_7X**: Performance benchmarks

6.4 Performance considerations and profiling

Memory layout:

- **Contiguous arrays**: All particle data stored in C-contiguous NumPy arrays,
- **Pool allocation**: Reduces dynamic memory allocation overhead,
- **Temporary arrays**: Field values cached at current time step.

Computational hotspots:

1. **Point Localization** (~30-40% of runtime)
2. **Field Interpolation** (~20-30% of runtime)
3. **Time Integration** (~15-25% of runtime)
4. **Random Number Generation** (~5-10% for diffusive cases)

Parallelization strategy:

- **Domain decomposition**: Not used (mesh not partitioned),
- **Particle decomposition**: Particles distributed among processes,
- **Communication**: Only for result merging at end.

To check the performance of the code after a modification, you can use the profiling options of Cython. To do so, you first need to activate profiling (and recompile):

```
./profile.sh 1
```

To deactivate profiling:

```
./profile.sh 0
```